

Лабораторная работа №3

Вопросы

- 1) Какую задачу выполняет таблица символов?
- 2) Что такое проходы компилятора? Для чего они нужны?
- 3) Какие есть готовые инструменты создания компиляторов?
- 4) Что такое язык? Какие виды языков бывают?
- 5) Что такое язык программирования? Из чего он состоит?
- 6) Что такое грамматика языка и какие её виды бывают?
- 7) Какие существуют типы грамматик по иерархии Хомского?
- 8) Что такое форма «Бэкуса - Наура» (БНФ)?
- 9) Из каких компонентов состоит БНФ?
- 10) Приведите пример описания инструкции языка программирования через БНФ.
- 11) Приведите пример описания грамматики арифметических выражений, состоящих из цифр и операций «+» и «-».
- 12) Приведите пример описания грамматики вызова функции.

Задание

Необходимо дополнить функционал консольной утилиты, созданной на предыдущей лабораторной работе, синтаксическим анализом, результатом работы которого является синтаксическое дерево.

Во-первых, после формирования последовательности токенов, утилита должна проверить синтаксис выражения, и сообщить об ошибке в случае их обнаружения.

Например:

Синтаксическая ошибка! У операции «*» на позиции 4 отсутствует операнд

Синтаксическая ошибка! У открывающей скобки «(» на позиции 15 отсутствует закрывающая скобка.

Синтаксическая ошибка! После идентификатора «id,2» на позиции 26 отсутствует операция.

Во-вторых, после проверки синтаксиса, в случае если ошибок не обнаружено, утилита должна построить синтаксическое дерево заданного выражения вида:

<id,1> <*> <(> <id,2> <+> <id,2> <*> <60> <)>



Все токены скобок приоритета операций при этом отбрасываются, тем самым приоритет операций задаётся порядком обхода дерева. Построение дерева следует осуществлять **БЕЗ** преобразования выражения в постфиксную форму.

В-третьих, утилита должна после формирования синтаксического дерева, визуализировать его в текстовый файл `syntax_tree.txt` с помощью `ascii` символов любым способом.

Например так:

```
<*>
|---<id, 1>
|---<+>
    |---<id, 2>
    |---<*>
        |---<id, 3>
        |---<60>
```

Или так:

```

      <*>
    /---|---\
<id, 1>      <+>
          /---|---\
        <id, 2>      <*>
                  /---|---\
                <id, 3>      <60>
```

Важно: утилита при вызове теперь должна дополнительно принимать первый параметр – режим работы: **LEX** или **lex** в случае запуска только лексического анализа, тогда генерируются файлы `tokens.txt` и `symbols.txt` как результат работы, **SYN** или **syn** в случае применения и лексического и синтаксического анализа, тогда генерируется только файл `syntax_tree.txt` как результат работы. Ошибки в консоль выводятся для всех фаз анализа, при возникновении ошибки работа утилиты прекращается.